

ECS 409/609

Assignment 8

Iterative Construct

Kunwar Arpit Singh

Submission Deadline: 09 November, 2025

For code repository of this assignment:  MIPS Assignments

1. Problem 1

The Fibonacci Sequence is the series of numbers:

0,1,1,2,3,5,8,13,21.....

that is produced by the adding up two number appear before the sequence. Write a MIPS assembly program that takes one number as an input, let say n and print the first n number of Fibonacci series by first storing into the memory. The output format(for n = 6) should be like this:

```
1 number of fibonacci series is stored at memory location 3000
2 number of fibonacci series is stored at memory location 3004
3 number of fibonacci series is stored at memory location 3008
.....
fibonacci series:0,1,1,2,3,5
```

MIPS Assembly Source

```
1 # Assignment 8: Iterative Construct
2 # Programmed By : Kunwar Arpit Singh
3
4 .data
5     prompt: .asciiz "Programmed By: Kunwar Arpit Singh\n\nEnter the number n: "
6     msg_part1: .asciiz "number of fibonacci series is stored at memory location "
7     msg_nl: .asciiz "\n"
8     msg_final: .asciiz "fibonacci series: "
9     msg_comma: .asciiz ","
10    .align 2
11    fib_array: .space 400
12 .text
13 .globl main
14 main:
15     li $v0, 4
```

```

16     la $a0, prompt
17     syscall
18
19     li $v0, 5
20     syscall
21     move $s0, $v0
22
23     la $s1, fib_array
24     li $s2, 1
25     li $t0, 0
26     li $t1, 1
27
28     beq $s0, $zero, exit
29
30 store_loop:
31     sgt $t7, $s2, $s0
32     bne $t7, $zero, print_series_header
33
34     beq $s2, 1, case_f1
35     beq $s2, 2, case_f2
36
37     add $t3, $t1, $t0
38     move $t0, $t1
39     move $t1, $t3
40     j store_and_print
41
42 case_f1:
43     li $t3, 0
44     j store_and_print
45
46 case_f2:
47     li $t3, 1
48
49 store_and_print:
50     sw $t3, 0($s1)
51
52     li $v0, 1
53     move $a0, $s2
54     syscall
55
56     li $v0, 4
57     la $a0, msg_part1
58     syscall
59
60     li $v0, 1
61     move $a0, $s1
62     syscall
63
64     li $v0, 4
65     la $a0, msg_nl
66     syscall

```

```

67
68     addi $s2, $s2, 1
69     addi $s1, $s1, 4
70
71     j store_loop
72
73 print_series_header:
74     li $v0, 4
75     la $a0, msg_final
76     syscall
77
78     la $s1, fib_array
79     li $t2, 0
80
81 print_loop:
82     beq $t2, $s0, exit
83
84     lw $a0, 0($s1)
85     li $v0, 1
86     syscall
87
88     addi $t2, $t2, 1
89     addi $s1, $s1, 4
90
91     beq $t2, $s0, exit
92
93     li $v0, 4
94     la $a0, msg_comma
95     syscall
96
97     j print_loop
98
99 exit:
100     li $v0, 10
101     syscall

```

Output Screenshots

Integer Register values and Console

File Simulator Registers Text Segment Data Segment			Programmed By: Runwar Arpit Singh
FP Regs	Int Regs [10]	#	Enter the number n: 48
PC	= 4194592	1	1 number of fibonacci series is stored at memory location 268501132
EPC	= 4194416	2	2 number of fibonacci series is stored at memory location 268501136
Cause	= 0	3	3 number of fibonacci series is stored at memory location 268501140
BadVAddr	= 0	4	4 number of fibonacci series is stored at memory location 268501144
Status	= 805371665	5	5 number of fibonacci series is stored at memory location 268501148
HI	= 0	6	6 number of fibonacci series is stored at memory location 268501152
LO	= 0	7	7 number of fibonacci series is stored at memory location 268501156
R0 [r0]	= 0	8	8 number of fibonacci series is stored at memory location 268501160
R1 [at]	= 268500992	9	9 number of fibonacci series is stored at memory location 268501164
R2 [v0]	= 10	10	10 number of fibonacci series is stored at memory location 268501168
R3 [v1]	= 0	11	11 number of fibonacci series is stored at memory location 268501172
R4	= 1836311903	12	12 number of fibonacci series is stored at memory location 268501176
R5 [a1]	= 2147479316	13	13 number of fibonacci series is stored at memory location 268501180
R6 [a2]	= 2147479336	14	14 number of fibonacci series is stored at memory location 268501184
R7 [a3]	= 0	15	15 number of fibonacci series is stored at memory location 268501188
R8 [t0]	= 1836311903	16	16 number of fibonacci series is stored at memory location 268501192
R9 [t1]	= 1836311903	17	17 number of fibonacci series is stored at memory location 268501196
R10 [t2]	= 48	18	18 number of fibonacci series is stored at memory location 268501200
R11 [t3]	= 1836311903	19	19 number of fibonacci series is stored at memory location 268501204
R12 [t4]	= 0	20	20 number of fibonacci series is stored at memory location 268501208
R13 [t5]	= 0	21	21 number of fibonacci series is stored at memory location 268501212
R14 [t6]	= 0	22	22 number of fibonacci series is stored at memory location 268501216
R15 [t7]	= 1	23	23 number of fibonacci series is stored at memory location 268501220
R16 [a0]	= 48	24	24 number of fibonacci series is stored at memory location 268501224
R17 [a1]	= 268501324	25	25 number of fibonacci series is stored at memory location 268501228
R18 [a2]	= 49	26	26 number of fibonacci series is stored at memory location 268501232
R19 [a3]	= 0	27	27 number of fibonacci series is stored at memory location 268501236
R20 [a4]	= 0	28	28 number of fibonacci series is stored at memory location 268501240
R21 [a5]	= 0	29	29 number of fibonacci series is stored at memory location 268501244
R22 [a6]	= 0	30	30 number of fibonacci series is stored at memory location 268501248
R23 [a7]	= 0	31	31 number of fibonacci series is stored at memory location 268501252
R24 [a8]	= 0	32	32 number of fibonacci series is stored at memory location 268501256
R25 [t9]	= 0	33	33 number of fibonacci series is stored at memory location 268501260
R26 [k0]	= 805371667	34	34 number of fibonacci series is stored at memory location 268501264
R27 [k1]	= 2	35	35 number of fibonacci series is stored at memory location 268501268
R28 [gp]	= 268468224	36	36 number of fibonacci series is stored at memory location 268501272
R29 [sp]	= 2147479312	37	37 number of fibonacci series is stored at memory location 268501276
R30 [s0]	= 0	38	38 number of fibonacci series is stored at memory location 268501280
R31 [ra]	= 4194328	39	39 number of fibonacci series is stored at memory location 268501284
Arithmetic overflow			40 number of fibonacci series is stored at memory location 268501288
			41 number of fibonacci series is stored at memory location 268501292
			42 number of fibonacci series is stored at memory location 268501296
			43 number of fibonacci series is stored at memory location 268501300
			44 number of fibonacci series is stored at memory location 268501304
			45 number of fibonacci series is stored at memory location 268501308
			46 number of fibonacci series is stored at memory location 268501312
			47 number of fibonacci series is stored at memory location 268501316
			Exception 12 [Arithmetic overflow] occurred and ignored
			48 number of fibonacci series is stored at memory location 268501320
			fibonacci series:
			0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2179309, 3524578, 5702887, 9227465, 14930352, 24157817, 39088169, 63245956, 102334155, 165580141, 267914296, 434944937, 701408739, 1134903170, 1836311903, 1836311903

Floating Point Register values and Console

File Simulator Registers Text Segment Data Segment			Programmed By: Runwar Arpit Singh
FP Regs	Int Regs [10]	#	Enter the number n: 48
FR	= 38912	1	1 number of fibonacci series is stored at memory location 268501132
PCSR	= 0	2	2 number of fibonacci series is stored at memory location 268501136
Single Precision			3 number of fibonacci series is stored at memory location 268501140
FP0	= 0.000000	4	4 number of fibonacci series is stored at memory location 268501144
FP1	= 0.000000	5	5 number of fibonacci series is stored at memory location 268501148
FP2	= 0.000000	6	6 number of fibonacci series is stored at memory location 268501152
FP3	= 0.000000	7	7 number of fibonacci series is stored at memory location 268501156
FP4	= 0.000000	8	8 number of fibonacci series is stored at memory location 268501160
FP5	= 0.000000	9	9 number of fibonacci series is stored at memory location 268501164
FP6	= 0.000000	10	10 number of fibonacci series is stored at memory location 268501168
FP7	= 0.000000	11	11 number of fibonacci series is stored at memory location 268501172
FP8	= 0.000000	12	12 number of fibonacci series is stored at memory location 268501176
FP9	= 0.000000	13	13 number of fibonacci series is stored at memory location 268501180
FP10	= 0.000000	14	14 number of fibonacci series is stored at memory location 268501184
FP11	= 0.000000	15	15 number of fibonacci series is stored at memory location 268501188
FP12	= 0.000000	16	16 number of fibonacci series is stored at memory location 268501192
FP13	= 0.000000	17	17 number of fibonacci series is stored at memory location 268501196
FP14	= 0.000000	18	18 number of fibonacci series is stored at memory location 268501200
FP15	= 0.000000	19	19 number of fibonacci series is stored at memory location 268501204
FP16	= 0.000000	20	20 number of fibonacci series is stored at memory location 268501208
FP17	= 0.000000	21	21 number of fibonacci series is stored at memory location 268501212
FP18	= 0.000000	22	22 number of fibonacci series is stored at memory location 268501216
FP19	= 0.000000	23	23 number of fibonacci series is stored at memory location 268501220
FP20	= 0.000000	24	24 number of fibonacci series is stored at memory location 268501224
FP21	= 0.000000	25	25 number of fibonacci series is stored at memory location 268501228
FP22	= 0.000000	26	26 number of fibonacci series is stored at memory location 268501232
FP23	= 0.000000	27	27 number of fibonacci series is stored at memory location 268501236
FP24	= 0.000000	28	28 number of fibonacci series is stored at memory location 268501240
FP25	= 0.000000	29	29 number of fibonacci series is stored at memory location 268501244
FP26	= 0.000000	30	30 number of fibonacci series is stored at memory location 268501248
FP27	= 0.000000	31	31 number of fibonacci series is stored at memory location 268501252
FP28	= 0.000000	32	32 number of fibonacci series is stored at memory location 268501256
FP29	= 0.000000	33	33 number of fibonacci series is stored at memory location 268501260
FP30	= 0.000000	34	34 number of fibonacci series is stored at memory location 268501264
FP31	= 0.000000	35	35 number of fibonacci series is stored at memory location 268501268
FP32	= 0.000000	36	36 number of fibonacci series is stored at memory location 268501272
FP33	= 0.000000	37	37 number of fibonacci series is stored at memory location 268501276
FP34	= 0.000000	38	38 number of fibonacci series is stored at memory location 268501280
FP35	= 0.000000	39	39 number of fibonacci series is stored at memory location 268501284
FP36	= 0.000000	40	40 number of fibonacci series is stored at memory location 268501288
FP37	= 0.000000	41	41 number of fibonacci series is stored at memory location 268501292
FP38	= 0.000000	42	42 number of fibonacci series is stored at memory location 268501296
FP39	= 0.000000	43	43 number of fibonacci series is stored at memory location 268501300
FP40	= 0.000000	44	44 number of fibonacci series is stored at memory location 268501304
FP41	= 0.000000	45	45 number of fibonacci series is stored at memory location 268501308
FP42	= 0.000000	46	46 number of fibonacci series is stored at memory location 268501312
FP43	= 0.000000	47	47 number of fibonacci series is stored at memory location 268501316
Double Precision			Exception 12 [Arithmetic overflow] occurred and ignored
FP0	= 0.000000	48	48 number of fibonacci series is stored at memory location 268501320
FP1	= 0.000000		fibonacci series:
FP2	= 0.000000		0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, 832040, 1346269, 2179309, 3524578, 5702887, 9227465, 14930352, 24157817, 39088169, 63245956, 102334155, 165580141, 267914296, 434944937, 701408739, 1134903170, 1836311903, 1836311903

2. Problem 2

Write an MIPS assembly program that lets the use to input an integer, then output a countdown from that number to 1 (inclusively), line by line. For example if user entered 4, a correct program would output the following sequence:

```
4
3
2
1
```

MIPS Assembly Source

```
1 # Assignment 8: Iterative Construct
2 # Programmed By : Kunwar Arpit Singh
3
4 .data
5     prompt: .asciiz "Programmed By: Kunwar Arpit Singh\n\nEnter a number:"
6     newline: .asciiz "\n"
7
8 .text
9 .globl main
10 main:
11     li $v0, 4
12     la $a0, prompt
13     syscall
14
15     li $v0, 5
16     syscall
17     move $s0, $v0
18
19     countdown_loop:
20         sgt $t0, $s0, $zero
21         beq $t0, $zero, exit
22
23         li $v0, 1
24         move $a0, $s0
25         syscall
26
27         li $v0, 4
28         la $a0, newline
29         syscall
30
31         addi $s0, $s0, -1
32
33         j countdown_loop
34
35     exit:
36         li $v0, 10
37         syscall
```

Output Screenshots

Integer Register values and Console

The screenshot displays the QtSpim simulator interface. The 'Registers' tab is active, showing the 'Int Regs [10]' window. The console window on the right shows the program's execution, including the prompt 'Enter a number: 57' and the output '57'.

Integer Register Values:

Register	Value
PC	4194412
EPC	0
Cause	0
BadVAddr	0
Status	805371664
HI	0
LO	0
R0 [r0]	0
R1 [at]	268500992
R2 [v0]	10
R3 [v1]	0
R4 [a0]	268501045
R5 [a1]	2147479316
R6 [a2]	2147479336
R7 [a3]	0
R8 [t0]	0
R9 [t1]	0
R10 [t2]	0
R11 [t3]	0
R12 [t4]	0
R13 [t5]	0
R14 [t6]	0
R15 [t7]	0
R16 [s0]	0
R17 [s1]	0
R18 [s2]	0
R19 [s3]	0
R20 [s4]	0
R21 [s5]	0
R22 [s6]	0
R23 [s7]	0
R24 [t8]	0
R25 [t9]	0
R26 [k0]	0
R27 [k1]	0
R28 [gp]	268468224
R29 [sp]	2147479312
R30 [s8]	0
R31 [ra]	4194328

Console Output:

```
Programmed By: Kunwar Arpit Singh
Enter a number: 57
57
```

Copyright 1990-2023 by James Larus.
All Rights Reserved.
SPIM is distributed under a BSD license.
See the file README for a full copyright notice.
QtSPIM is linked to the Qt library, which is

Floating Point Register values and Console

QtSpim
File Simulator Registers Text Segment Data Segment
FP Regs Int Regs [10]

FP Regs
FIR = 38912
FCSR = 0

Single Precision
FG0 = 0.000000
FG1 = 0.000000
FG2 = 0.000000
FG3 = 0.000000
FG4 = 0.000000
FG5 = 0.000000
FG6 = 0.000000
FG7 = 0.000000
FG8 = 0.000000
FG9 = 0.000000
FG10 = 0.000000
FG11 = 0.000000
FG12 = 0.000000
FG13 = 0.000000
FG14 = 0.000000
FG15 = 0.000000
FG16 = 0.000000
FG17 = 0.000000
FG18 = 0.000000
FG19 = 0.000000
FG20 = 0.000000
FG21 = 0.000000
FG22 = 0.000000
FG23 = 0.000000
FG24 = 0.000000
FG25 = 0.000000
FG26 = 0.000000
FG27 = 0.000000
FG28 = 0.000000
FG29 = 0.000000
FG30 = 0.000000
FG31 = 0.000000

Double Precision
FP0 = 0.000000
FP2 = 0.000000

Copyright 1990-2023 by James Larus.
All Rights Reserved.
SPIM is distributed under a BSD license.
See the file README for a full copyright notice.
QtSPIM is linked to the Qt library, which is

Console
Programmed By: Kunwar Arpit Singh
Enter a number: 57
57
56
55
54
53
52
51
50
49
48
47
46
45
44
43
42
41
40
39
38
37
36
35
34
33
32
31
30
29
28
27
26
25
24
23
22
21
20
19
18
17
16
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1

3. Problem 3

Problem statement

Write a complete MIPS program that implements the algorithm shown below (in C):

```
void main()
{
    int J = 1;
    int a;
    a = 0;

    while(1) {
        if(a > 20)
            break;
        J++;
        a+=2;
    }

    do {
        J++;
    } while (J < 1000);
}

while (a > 0){
    J--;
    a--;
}
```

MIPS Assembly Source

```
1  # Assignment 8: Iterative Construct
2  # Programmed By : Kunwar Arpit Singh
3
4  .data
5      msg_J: .asciiz "Programmed By: Kunwar Arpit Singh\n\nFinal J:"
6      msg_a: .asciiz "Final a:"
7      newline: .asciiz "\n"
8
9  .text
10 .globl main
11 main:
12     li $s0, 1
13
14     li $s1, 0
15
16 loop1_start:
17     li $t0, 20
18     sgt $t1, $s1, $t0
19
```

```

20     bne $t1, $zero, loop1_end
21
22     addi $s0, $s0, 1
23
24     addi $s1, $s1, 2
25
26     j loop1_start
27
28 loop1_end:
29
30 loop2_start:
31     addi $s0, $s0, 1
32
33     li $t0, 100
34     slt $t1, $s0, $t0
35     bne $t1, $zero, loop2_start
36
37 loop2_end:
38
39 loop3_start:
40     sgt $t1, $s1, $zero
41     beq $t1, $zero, loop3_end
42
43     addi $s0, $s0, -1
44
45     addi $s1, $s1, -1
46
47     j loop3_start
48
49 loop3_end:
50
51     li $v0, 4
52     la $a0, msg_J
53     syscall
54
55     li $v0, 1
56     move $a0, $s0
57     syscall
58
59     li $v0, 4
60     la $a0, newline
61     syscall
62
63     li $v0, 4
64     la $a0, msg_a
65     syscall
66
67     li $v0, 1
68     move $a0, $s1
69     syscall
70

```

```

71     li $v0, 10
72     syscall

```

Output Screenshots

Integer Register values and Console

The screenshot displays a debugger interface with two main panes. The left pane, titled 'Int Regs [10]', shows the values of various registers. The right pane, titled 'Console', shows the output of the program.

Integer Register Values:

Register	Value
PC	4194480
EPC	0
Cause	0
BadVAddr	0
Status	805371664
HI	0
LO	0
R0 [r0]	0
R1 [at]	268500992
R2 [v0]	10
R3 [v1]	0
R4 [a0]	0
R5 [a1]	2147479316
R6 [a2]	2147479336
R7 [a3]	0
R8 [t0]	100
R9 [t1]	0
R10 [t2]	0
R11 [t3]	0
R12 [t4]	0
R13 [t5]	0
R14 [t6]	0
R15 [t7]	0
R16 [s0]	78
R17 [s1]	0
R18 [s2]	0
R19 [s3]	0
R20 [s4]	0
R21 [s5]	0
R22 [s6]	0
R23 [s7]	0
R24 [t8]	0
R25 [t9]	0
R26 [k0]	0
R27 [k1]	0
R28 [gp]	268468224
R29 [sp]	2147479312
R30 [s8]	0
R31 [ra]	4194328

Console Output:

```

Programmed By: Kunwar Arpit Singh

Final J: 78
Final a: 0

```

Floating Point Register values and Console

The screenshot shows a debugger window with two main panes. The left pane, titled 'FP Regs', displays the values of floating point registers. The right pane, titled 'Text', shows the console output.

FP Regs

FIR = 38912
FCSR = 0

Single Precision

FG0 = 0.000000
FG1 = 0.000000
FG2 = 0.000000
FG3 = 0.000000
FG4 = 0.000000
FG5 = 0.000000
FG6 = 0.000000
FG7 = 0.000000
FG8 = 0.000000
FG9 = 0.000000
FG10 = 0.000000
FG11 = 0.000000
FG12 = 0.000000
FG13 = 0.000000
FG14 = 0.000000
FG15 = 0.000000
FG16 = 0.000000
FG17 = 0.000000
FG18 = 0.000000
FG19 = 0.000000
FG20 = 0.000000
FG21 = 0.000000
FG22 = 0.000000
FG23 = 0.000000
FG24 = 0.000000
FG25 = 0.000000
FG26 = 0.000000
FG27 = 0.000000
FG28 = 0.000000
FG29 = 0.000000
FG30 = 0.000000
FG31 = 0.000000

Double Precision

FP0 = 0.000000
FP2 = 0.000000
FP4 = 0.000000
FP6 = 0.000000
FP8 = 0.000000
FP10 = 0.000000
FP12 = 0.000000
FP14 = 0.000000
FP16 = 0.000000
FP18 = 0.000000
FP20 = 0.000000
FP22 = 0.000000
FP24 = 0.000000
FP26 = 0.000000
FP28 = 0.000000
FP30 = 0.000000

Console

Programmed By: Kunwar Arpit Singh

Final J: 78
Final a: 0

4. Problem 4

Problem statement

Write an MIPS assembly program that let the user to input as a integer number, let say k and prompt the following output in the screen if the value of $k = 4$

```
1
121
12321
1234321
```

MIPS Assembly Source

```
1 # Assignment 8: Iterative Construct
2 # Programmed By : Kunwar Arpit Singh
3
4 .data
5     prompt: .asciiz "Programmed By: Kunwar Arpit Singh\n\nEnter k:"
6     newline: .asciiz "\n"
7     space: .asciiz " "
8 .text
9 .globl main
10 main:
11
12     li $v0, 4
13     la $a0, prompt
14     syscall
15
16     li $v0, 5
17     syscall
18     move $s0, $v0
19
20     li $s1, 1
21
22 outer_loop_start:
23     sgt $t0, $s1, $s0
24     bne $t0, $zero, exit
25
26     li $s3, 0
27     sub $t1, $s0, $s1
28
29 space_loop_start:
30     bge $s3, $t1, inner_loop_1_start
31
32     li $v0, 4
33     la $a0, space
34     syscall
35
36     addi $s3, $s3, 1
37     j space_loop_start
```

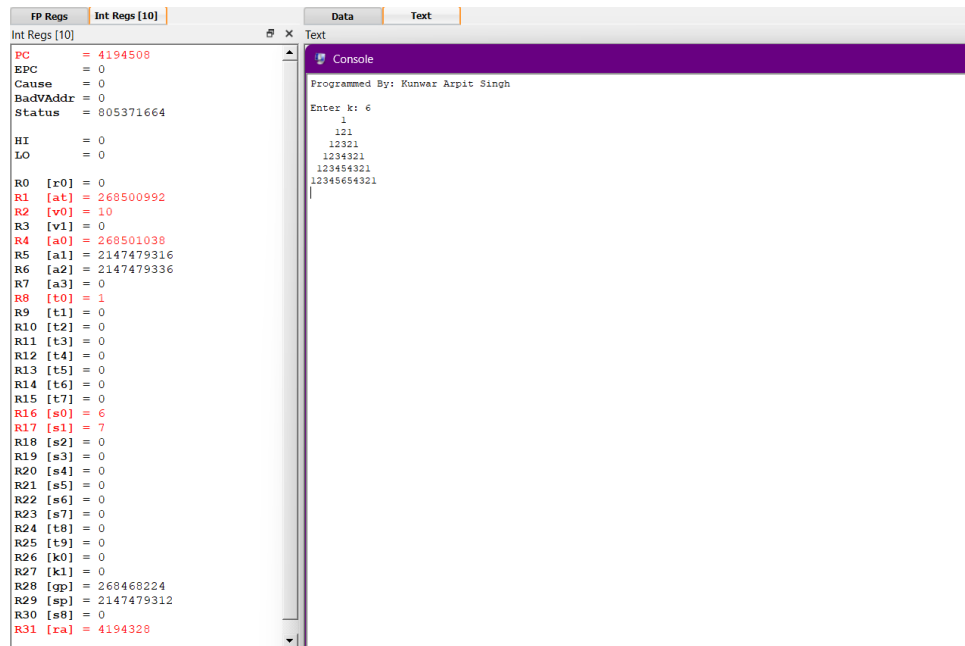
```

38
39 inner_loop_1_start:
40     li $s2, 1
41
42 inner_loop_1:
43     sgt $t0, $s2, $s1
44     bne $t0, $zero, inner_loop_2_start
45
46     li $v0, 1
47     move $a0, $s2
48     syscall
49
50     addi $s2, $s2, 1
51
52     j inner_loop_1
53
54 inner_loop_2_start:
55     addi $s2, $s1, -1 # j = i - 1
56
57 inner_loop_2:
58     slt $t0, $s2, 1 # $t0 = (j < 1)
59     bne $t0, $zero, print_newline
60
61     li $v0, 1
62     move $a0, $s2
63     syscall
64
65     addi $s2, $s2, -1
66
67     j inner_loop_2
68
69 print_newline:
70     li $v0, 4
71     la $a0, newline
72     syscall
73
74     addi $s1, $s1, 1 # i++
75
76     j outer_loop_start
77
78 exit:
79     li $v0, 10
80     syscall

```

Output Screenshots

Integer Register values and Console



Floating Point Register values and Console

